INSTITUTE OF SPACE TECHNOLOGY
KICSIT, Kahuta Campus

Code Snippets and Detailed Explanation

AI-Based Food Recognition Android Application

Submitted By

Safa Waseem – 242201054

Course
Mobile App Development – Practical

Semester
BSCS 5-B

Code Snippets
AI-Based Android Application

Submitted To
Sir Uzair Hassan

Department of Computer Science
KICSIT, IST Kahuta Campus

October 30, 2026

Contents

1 Introduction **3**

2 Project Directory Structure **3**

3 Application Workflow **3**

4 Android Application Source Code **4**

4.1 SplashActivity.java 4

4.2 LoginActivity.java 5

4.3 MainActivity.java 7

4.4 CameraActivity.java 8

5 FoodClassifier.java – TensorFlow Lite **12**

5.1 Introduction 12

5.2 Model Files 12

5.3 Model Loading 12

5.4 Detailed Explanation 14

5.5 TensorFlow Lite Inference Workflow 15

6 Model Training Using Python **15**

6.1 Introduction 15

6.2 Training Environment 16

6.3 Dataset Preparation 16

7 ResultActivity.java **17**

7.1 Purpose 17

7.2 Detailed Explanation 19

8 AndroidManifest.xml **19**

9 Gradle Dependencies **20**

9.1 Purpose 20

10 XML Layout Files **21**

10.1 Main Screen Layout 21

10.2 Camera Screen Layout 25

10.3 Result Screen Layout 27

10.4 Screenshots Required for the Final Report 31

11 Training Results **38**

12 Testing and Discussion **39**

13 Conclusion **39**

14 References **39**

1 Introduction

FoodAI is an AI-based Android application developed to recognize food items from images using Machine Learning and TensorFlow Lite.

The application allows users to capture a food image using the camera or select an existing image from the device gallery.

The selected image is processed by a trained machine learning model. The model predicts the food category and returns a confidence score.

The application also displays nutritional information such as calories, carbohydrates, protein, and fat.

This document presents the actual source code, model integration, training implementation, and application output of FoodAI.

2 Project Directory Structure

The main components of the FoodAI project are:

```
FoodAI/
|
|-- app/
|   |-- src/
|       |-- main/
|           |-- java/com/example/foodai/
|           |   |-- SplashActivity.java
|           |   |-- LoginActivity.java
|           |   |-- MainActivity.java
|           |   |-- CameraActivity.java
|           |   |-- FoodClassifier.java
|           |   |-- ResultActivity.java
|           |
|           |-- assets/
|           |   |-- model.tflite
|           |   |-- labels.txt
|           |
|           |-- res/layout/
|               |-- activity_main.xml
|               |-- activity_camera.xml
|               |-- activity_result.xml
|
|-- build.gradle
|-- settings.gradle
|-- gradle/
```

Explanation:

The Java directory contains the application's main activities and classification logic.

The assets directory stores the trained TensorFlow Lite model and its corresponding labels.

The XML layout directory contains the user interface designs of the application.

3 Application Workflow

The FoodAI application follows these steps:

1. The application starts with the splash screen.
2. The user enters the application through login.
3. The home screen provides the food scanning option.

4. The user captures an image or selects one from the gallery.
5. The image is resized and converted into the model's input format.
6. TensorFlow Lite performs on-device inference.
7. The highest prediction score identifies the food category.
8. The result screen displays the predicted food and nutritional information.

4 Android Application Source Code

4.1 SplashActivity.java

Purpose:

SplashActivity is the starting screen of FoodAI. It displays the splash interface before navigating to the login screen.

Important functionality:

- Initializes the splash screen.
- Loads the splash XML layout.
- Uses a Handler to delay navigation.
- Opens LoginActivity.

Complete Source Code:

Listing 1: SplashActivity.java

```
1
2  package com.example.foodai;
3
4  import android.content.Intent;
5  import android.os.Bundle;
6  import android.os.Handler;
7  import android.os.Looper;
8
9  import androidx.appcompat.app.AppCompatActivity;
10
11 public class SplashActivity extends AppCompatActivity {
12
13     @Override
14     protected void onCreate(Bundle savedInstanceState) {
15         super.onCreate(savedInstanceState);
16         setContentView(R.layout.activity_splash);
17
18         new Handler(Looper.getMainLooper()).postDelayed(() -> {
19             Intent intent = new Intent(
20                     SplashActivity.this,
21                     LoginActivity.class
22             );
23             startActivity(intent);
24             finish();
25         }, 2500);
26     }
27 }
```

Detailed Explanation:

The onCreate() method is called when Android creates the activity.

The setContentView() method connects the Java activity with its XML layout.

Handler and postDelayed() are used to create a delay before opening the next activity.

An Intent is used to navigate to LoginActivity. The finish() method closes the splash activity.

4.2 LoginActivity.java

Purpose:

LoginActivity provides the login and sign-up functionality of the FoodAI application.

Important functionality:

- Reads user input.
- Validates the entered information.
- Stores demo login information using SharedPreferences.
- Navigates to MainActivity after login.

Complete Source Code:

Listing 2: LoginActivity.java

```java
package com.example.foodai;

import android.content.Intent;
import android.content.SharedPreferences;
import android.os.Bundle;
import android.util.Patterns;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

public class LoginActivity extends AppCompatActivity {

    EditText nameInput, emailInput, passwordInput;
    TextView titleText, subtitleText, switchModeText;
    Button actionButton;

    boolean isSignUp = false;

    SharedPreferences preferences;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_login);

        nameInput = findViewById(R.id.nameInput);
        emailInput = findViewById(R.id.emailInput);
        passwordInput = findViewById(R.id.passwordInput);
        titleText = findViewById(R.id.titleText);
        subtitleText = findViewById(R.id.subtitleText);
        switchModeText = findViewById(R.id.switchModeText);
        actionButton = findViewById(R.id.actionButton);

        preferences = getSharedPreferences(
                "FoodAIPreferences", MODE_PRIVATE);

        actionButton.setOnClickListener(v -> {
            if (isSignUp) {
                signUp();
            } else {
                login();
            }
```

```
        });

        switchModeText.setOnClickListener(v -> {
            isSignUp = !isSignUp;
            updateScreen();
        });
    }

    private void updateScreen() {
        if (isSignUp) {
            titleText.setText("Create Account");
            subtitleText.setText(
                    "Join FoodAI and start your healthy journey.");
            actionButton.setText("Sign Up");
            switchModeText.setText(
                    "Already have an account? Login");
            nameInput.setVisibility(View.VISIBLE);
        } else {
            titleText.setText("Welcome Back!");
            subtitleText.setText(
                    "Login to continue your healthy journey.");
            actionButton.setText("Login");
            switchModeText.setText(
                    "Don't have an account? Sign Up");
            nameInput.setVisibility(View.GONE);
        }
    }

    private void signUp() {
        String name = nameInput.getText().toString().trim();
        String email = emailInput.getText().toString()
                .trim().toLowerCase();
        String password = passwordInput.getText().toString();

        if (name.isEmpty()) {
            nameInput.setError("Enter your name");
            nameInput.requestFocus();
            return;
        }

        if (!Patterns.EMAIL_ADDRESS.matcher(email).matches()) {
            emailInput.setError("Enter a valid email");
            emailInput.requestFocus();
            return;
        }

        if (password.length() < 6) {
            passwordInput.setError(
                    "Password must be at least 6 characters");
            passwordInput.requestFocus();
            return;
        }

        // Demo account storage for this device only
        preferences.edit()
                .putString("name", name)
                .putString("email", email)
                .putString("password", password)
                .apply();

        Toast.makeText(this,
                "Account created successfully!",
                Toast.LENGTH_SHORT).show();

        openMainScreen();
    }

    private void login() {
        String email = emailInput.getText().toString()
                .trim().toLowerCase();
        String password = passwordInput.getText().toString();

        if (email.isEmpty()) {
            emailInput.setError("Enter your email");
            emailInput.requestFocus();
            return;
```

```
        }

        if (password.isEmpty()) {
            passwordInput.setError("Enter your password");
            passwordInput.requestFocus();
            return;
        }

        String savedEmail = preferences.getString("email", "");
        String savedPassword =
                preferences.getString("password", "");

        if (email.equals(savedEmail)
                && password.equals(savedPassword)
                && !savedEmail.isEmpty()) {

            Toast.makeText(this,
                    "Login successful!",
                    Toast.LENGTH_SHORT).show();

            openMainScreen();

        } else {
            Toast.makeText(this,
                    "Account not found or password incorrect. Sign Up first.",
                    Toast.LENGTH_LONG).show();
        }
    }

    private void openMainScreen() {
        Intent intent = new Intent(
                LoginActivity.this, MainActivity.class);
        startActivity(intent);
        finish();
    }
}
```

Detailed Explanation:

The EditText components collect user information.

SharedPreferences stores simple key-value data locally on the device.

The sign-up method validates the entered information before saving it.

The login method checks the entered credentials against the locally saved demo credentials.

4.3 MainActivity.java

Purpose:

MainActivity represents the home screen of FoodAI.

It provides navigation to the food scanning functionality.

Complete Source Code:

Listing 3: MainActivity.java

```
package com.example.foodai;

import android.content.Intent;
import android.os.Bundle;
import android.widget.Button;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
```

```
        setContentView(R.layout.activity_main);

        Button btnScan = findViewById(R.id.btnScan);
        Button btnManual = findViewById(R.id.btnManual);

        // "Let's scan" dabane par Camera screen khulegi
        btnScan.setOnClickListener(v ->
                startActivity(new Intent(MainActivity.this, CameraActivity.class)));

        // Manual logging abhi ke liye sirf message
        btnManual.setOnClickListener(v ->
                Toast.makeText(this, "Manual log jald aayega", Toast.LENGTH_SHORT).show());
    }
}
```

Detailed Explanation:

The onCreate() method initializes the home screen.

findViewById() connects Java variables with buttons defined in the XML layout.

The scan button uses an Intent to open CameraActivity.

4.4 CameraActivity.java

Purpose:

CameraActivity manages camera capture and gallery image selection.

It uses CameraX libraries to access the device camera.

Important functionality:

- Requests camera permission.
- Displays camera preview.
- Captures food images.
- Allows gallery image selection.
- Passes the selected image to FoodClassifier.

Complete Source Code:

Listing 4: CameraActivity.java

```

package com.example.foodai;

import android.Manifest;
import android.content.Intent;
import android.content.pm.PackageManager;
import android.graphics.Bitmap;
import android.graphics.BitmapFactory;
import android.net.Uri;
import android.os.Bundle;
import android.view.View;
import android.widget.ImageButton;
import android.widget.Toast;

import androidx.activity.result.ActivityResultLauncher;
import androidx.activity.result.contract.ActivityResultContracts;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.camera.core.Camera;
import androidx.camera.core.CameraSelector;
import androidx.camera.core.ImageCapture;
import androidx.camera.core.ImageCaptureException;
```

```
import androidx.camera.core.Preview;
import androidx.camera.lifecycle.ProcessCameraProvider;
import androidx.camera.view.PreviewView;
import androidx.core.content.ContextCompat;

import com.google.common.util.concurrent.ListenableFuture;

import java.io.File;
import java.io.IOException;
import java.io.InputStream;
import java.util.concurrent.ExecutionException;

public class CameraActivity extends AppCompatActivity {

    private PreviewView previewView;
    private ImageCapture imageCapture;
    private Camera camera;
    private boolean flashOn = false;

    private final ActivityResultLauncher<String> permissionLauncher =
            registerForActivityResult(
                    new ActivityResultContracts.RequestPermission(),
                    granted -> {
                        if (granted) {
                            startCamera();
                        } else {
                            Toast.makeText(this,
                                    "Camera permission zaroori hai",
                                    Toast.LENGTH_LONG).show();
                            finish();
                        }
                    });

    private final ActivityResultLauncher<String> galleryLauncher =
            registerForActivityResult(
                    new ActivityResultContracts.GetContent(),
                    this::onImagePicked);

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_camera);

        previewView = findViewById(R.id.previewView);

        ImageButton btnBack = findViewById(R.id.btnBack);
        ImageButton btnCapture = findViewById(R.id.btnCapture);
        View btnGallery = findViewById(R.id.btnGallery);
        View btnFlash = findViewById(R.id.btnFlash);

        btnBack.setOnClickListener(v -> finish());
        btnCapture.setOnClickListener(v -> takePhoto());
        btnGallery.setOnClickListener(
                v -> galleryLauncher.launch("image/*"));
        btnFlash.setOnClickListener(v -> toggleFlash());

        if (ContextCompat.checkSelfPermission(
                this, Manifest.permission.CAMERA)
                == PackageManager.PERMISSION_GRANTED) {
            startCamera();
        } else {
            permissionLauncher.launch(Manifest.permission.CAMERA);
        }
    }

    private void startCamera() {
        ListenableFuture<ProcessCameraProvider> future =
                ProcessCameraProvider.getInstance(this);

        future.addListener(() -> {
            try {
                ProcessCameraProvider provider = future.get();

                Preview preview = new Preview.Builder().build();
                preview.setSurfaceProvider(
                        previewView.getSurfaceProvider());
```

```
                imageCapture = new ImageCapture.Builder().build();

                provider.unbindAll();

                camera = provider.bindToLifecycle(
                        this,
                        CameraSelector.DEFAULT_BACK_CAMERA,
                        preview,
                        imageCapture);

            } catch (ExecutionException | InterruptedException e) {
                Toast.makeText(this,
                        "Camera error: " + e.getMessage(),
                        Toast.LENGTH_LONG).show();
            }
        }, ContextCompat.getMainExecutor(this));
    }

    private void toggleFlash() {
        if (camera != null
                && camera.getCameraInfo().hasFlashUnit()) {
            flashOn = !flashOn;
            camera.getCameraControl().enableTorch(flashOn);
        } else {
            Toast.makeText(this,
                    "Flash maujood nahi",
                    Toast.LENGTH_SHORT).show();
        }
    }

    // Camera se photo lena aur classify karna
    private void takePhoto() {
        if (imageCapture == null) {
            Toast.makeText(this,
                    "Camera abhi ready nahi hai",
                    Toast.LENGTH_SHORT).show();
            return;
        }

        File photoFile = new File(
                getCacheDir(),
                "scan_" + System.currentTimeMillis() + ".jpg");

        ImageCapture.OutputFileOptions options =
                new ImageCapture.OutputFileOptions.Builder(
                        photoFile).build();

        imageCapture.takePicture(
                options,
                ContextCompat.getMainExecutor(this),
                new ImageCapture.OnImageSavedCallback() {

                    @Override
                    public void onImageSaved(
                            @NonNull ImageCapture.OutputFileResults results) {

                        Bitmap bitmap = BitmapFactory.decodeFile(
                                photoFile.getAbsolutePath());

                        if (bitmap == null) {
                            Toast.makeText(CameraActivity.this,
                                    "Photo open nahi ho saki",
                                    Toast.LENGTH_LONG).show();
                            return;
                        }

                        classifyAndShowResult(
                                bitmap,
                                photoFile.getAbsolutePath(),
                                null);
                    }

                    @Override
                    public void onError(
                            @NonNull ImageCaptureException e) {
```

```
                        Toast.makeText(CameraActivity.this,
                                "Capture fail: " + e.getMessage(),
                                Toast.LENGTH_LONG).show();
                    }
                });
    }

    // Gallery se select ki hui image
    private void onImagePicked(Uri uri) {
        if (uri == null) {
            return;
        }

        Bitmap bitmap;

        try (InputStream inputStream =
                     getContentResolver().openInputStream(uri)) {

            if (inputStream == null) {
                Toast.makeText(this,
                        "Image open nahi hui",
                        Toast.LENGTH_LONG).show();
                return;
            }

            bitmap = BitmapFactory.decodeStream(inputStream);

        } catch (IOException e) {
            Toast.makeText(this,
                    "Image open error: " + e.getMessage(),
                    Toast.LENGTH_LONG).show();
            return;
        }

        if (bitmap == null) {
            Toast.makeText(this,
                    "Image read nahi ho saki",
                    Toast.LENGTH_LONG).show();
            return;
        }

        classifyAndShowResult(bitmap, null, uri.toString());
    }

    // Camera aur gallery dono ka common prediction method
    private void classifyAndShowResult(
            Bitmap bitmap, String imagePath, String imageUri) {

        FoodClassifier classifier = null;

        try {
            classifier = new FoodClassifier(this);

            FoodClassifier.Result result =
                    classifier.classify(bitmap);

            Intent intent = new Intent(
                    CameraActivity.this,
                    ResultActivity.class);

            intent.putExtra(
                    ResultActivity.EXTRA_LABEL,
                    result.label);

            intent.putExtra(
                    ResultActivity.EXTRA_CONFIDENCE,
                    result.confidence);

            if (imagePath != null) {
                intent.putExtra(
                        ResultActivity.EXTRA_IMAGE_PATH,
                        imagePath);
            }

            if (imageUri != null) {
                intent.putExtra(
```

```
                        ResultActivity.EXTRA_IMAGE_URI,
                        imageUri);
            }

            startActivity(intent);

        } catch (Exception e) {
            Toast.makeText(this,
                    "Prediction error: " + e.getMessage(),
                    Toast.LENGTH_LONG).show();

        } finally {
            if (classifier != null) {
                classifier.close();
            }

            if (!bitmap.isRecycled()) {
                bitmap.recycle();
            }
        }
    }
}
```

Detailed Explanation:

CameraX provides camera preview and image capture functionality.

The permission launcher requests access to the device camera.

The gallery launcher allows users to select an existing image.

After receiving the image, the application passes its Bitmap to FoodClassifier for prediction.

5 FoodClassifier.java – TensorFlow Lite

5.1 Introduction

FoodClassifier is the main machine learning component of the FoodAI application.

It loads the trained TensorFlow Lite model, prepares image input, performs inference, and returns the predicted food category.

5.2 Model Files

The application uses the following files:

- `model.tflite`: Trained food recognition model.
- `labels.txt`: Contains food category labels.

The model is stored inside the Android assets folder.

5.3 Model Loading

The TensorFlow Lite Interpreter loads the trained model into memory.

Complete Source Code:

Listing 5: FoodClassifier.java

```
package com.example.foodai;

```

```java
import android.content.Context;
import android.graphics.Bitmap;

import org.tensorflow.lite.Interpreter;

import java.io.BufferedReader;
import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.nio.ByteBuffer;
import java.nio.ByteOrder;
import java.util.ArrayList;
import java.util.List;

/**
 * FoodClassifier: TensorFlow Lite model ko app ke andar chalata hai.
 *
 * Kaam ka tareeqa:
 *   1. model.tflite aur labels.txt "assets" folder se load hoti hain
 *   2. Photo ko 224x224 size mein badla jata hai (model ka input size)
 *   3. Photo ke pixels ko ek buffer mein rakha jata hai
 *   4. Interpreter model chalata hai aur har class ki probability deta hai
 *   5. Sab se zyada probability wala naam wapis diya jata hai
 */
public class FoodClassifier {

    // Assets folder mein files ke naam
    private static final String MODEL_FILE = "model.tflite";
    private static final String LABEL_FILE = "labels.txt";

    // MobileNetV2 ka input size 224 x 224 pixels hai (Colab mein bhi yahi tha)
    private static final int INPUT_SIZE = 224;

    // Interpreter: TFLite ka wo object jo model ko chalata hai
    private final Interpreter interpreter;

    // Class names (apple, banana, ...). Order wahi hai jo Colab ki labels.txt mein tha
    private final List<String> labels;

    /** Prediction ka natija: food ka naam aur kitna yaqeen (0 se 1 tak) */
    public static class Result {
        public final String label;
        public final float confidence;

        public Result(String label, float confidence) {
            this.label = label;
            this.confidence = confidence;
        }
    }

    /** Model aur labels load karta hai. Agar file na mile to IOException aata hai. */
    public FoodClassifier(Context context) throws IOException {
        interpreter = new Interpreter(loadModel(context));
        labels = loadLabels(context);
    }

    /** assets se model.tflite padh kar ByteBuffer mein deta hai (Interpreter ko yahi chahiye). */
    private ByteBuffer loadModel(Context context) throws IOException {
        try (InputStream in = context.getAssets().open(MODEL_FILE)) {
            ByteArrayOutputStream out = new ByteArrayOutputStream();
            byte[] chunk = new byte[8192];
            int n;
            while ((n = in.read(chunk)) != -1) {
                out.write(chunk, 0, n);
            }
            byte[] bytes = out.toByteArray();

            // "Direct" buffer aur native byte order: TFLite ki zaroorat
            ByteBuffer buffer = ByteBuffer.allocateDirect(bytes.length);
            buffer.order(ByteOrder.nativeOrder());
            buffer.put(bytes);
            buffer.rewind();
            return buffer;
        }
    }
```

```
79
80      /** assets se labels.txt padhta hai. Har line mein ek class ka naam hota hai. */
81      private List<String> loadLabels(Context context) throws IOException {
82          List<String> list = new ArrayList<>();
83          try (BufferedReader reader = new BufferedReader(
84                  new InputStreamReader(context.getAssets().open(LABEL_FILE)))) {
85              String line;
86              while ((line = reader.readLine()) != null) {
87                  if (!line.trim().isEmpty()) {
88                      list.add(line.trim());
89                  }
90              }
91          }
92          return list;
93      }
94
95      /** Ek photo (Bitmap) ko pehchanta hai aur sab se behtar natija wapis deta hai. */
96      public Result classify(Bitmap source) {
97          // Step 1: Photo ko 224 x 224 mein badlo
98          Bitmap bitmap = Bitmap.createScaledBitmap(source, INPUT_SIZE, INPUT_SIZE, true);
99
100         // Step 2: Input buffer banao.
101         // Shape [1, 224, 224, 3], har pixel ke 3 float numbers (R, G, B).
102         // 4 bytes har float ke liye.
103         ByteBuffer input = ByteBuffer.allocateDirect(4 * INPUT_SIZE * INPUT_SIZE * 3);
104         input.order(ByteOrder.nativeOrder());
105
106         int[] pixels = new int[INPUT_SIZE * INPUT_SIZE];
107         bitmap.getPixels(pixels, 0, INPUT_SIZE, 0, 0, INPUT_SIZE, INPUT_SIZE);
108
109         for (int pixel : pixels) {
110             // Pixel ek int hota hai jisme Alpha, Red, Green, Blue bhare hote hain
111             float r = (pixel >> 16) & 0xFF;
112             float g = (pixel >> 8) & 0xFF;
113             float b = pixel & 0xFF;
114
115             // Colab mein normalization (0..255 se -1..1) model ke ANDAR shamil thi,
116             // is liye yahan seedha 0..255 wali values dete hain.
117             input.putFloat(r);
118             input.putFloat(g);
119             input.putFloat(b);
120         }
121
122         // Step 3: Output array: [1][number_of_classes]. Har class ki probability.
123         float[][] output = new float[1][labels.size()];
124
125         // Step 4: Model chalao (inference)
126         interpreter.run(input, output);
127
128         // Step 5: Sab se bari probability dhoondo
129         int best = 0;
130         for (int i = 1; i < output[0].length; i++) {
131             if (output[0][i] > output[0][best]) {
132                 best = i;
133             }
134         }
135         return new Result(labels.get(best), output[0][best]);
136     }
137
138     /** Kaam khatam hone par memory wapis lene ke liye */
139     public void close() {
140         interpreter.close();
141     }
142 }
```

5.4 Detailed Explanation

1. Model Initialization

The Interpreter initializes the trained model from the model buffer.

2. Loading Labels

The labels.txt file is read line by line. Each label represents a food category.

3. Image Preprocessing

The input image is resized to 224 by 224 pixels.

RGB pixel values are extracted and converted into the required input buffer.

4. Model Inference

The Interpreter.run() method executes the model.

The input image is processed and prediction scores are generated.

5. Prediction Selection

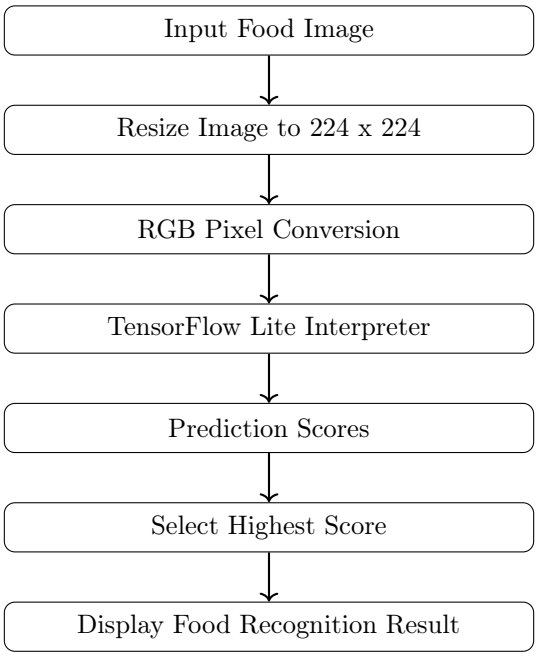
The application searches for the highest prediction score.

The corresponding label is selected as the predicted food category.

6. Resource Management

The close() method releases Interpreter resources after classification.

5.5 TensorFlow Lite Inference Workflow



6 Model Training Using Python

6.1 Introduction

The food recognition model is trained using machine learning techniques.

Training involves providing food images to the model so that it can learn visual features and distinguish between different food categories.

The trained model is exported into TensorFlow Lite format for Android integration.

6.2 Training Environment

- Programming Language: Python
- Framework: TensorFlow / Keras
- Dataset: Actual training dataset used
- Model Format: TensorFlow Lite
- Input Image Size: 224 x 224

6.3 Dataset Preparation

Explanation:

The dataset contains food images organized according to their respective classes.

The images are loaded and prepared for training. Preprocessing ensures that images have the required dimensions and format.

Complete Python Training Code:

Listing 6: FoodAI Model Training Code

```
# -*- coding: utf-8 -*-
"""Untitled0.ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/1YOz-1ovfveZQO2d016ANk_exmI55WQ_W
"""

import os, shutil, random, glob

!git clone --depth 1 https://github.com/Horea94/Fruit-Images-Dataset.git

SRC = "Fruit-Images-Dataset"
CLASSES = ["apple", "banana", "orange", "strawberry", "tomato", "potato", "mango", "grape"]

def collect(split_dir, out_dir, limit):
    for cls in CLASSES:
        os.makedirs(f"{out_dir}/{cls}", exist_ok=True)
        files = []
        for folder in os.listdir(split_dir):
            f = folder.lower()
            if f == cls or f.startswith(cls + " "):
                files += glob.glob(f"{split_dir}/{folder}/*.jpg")
        random.shuffle(files)
        for i, path in enumerate(files[:limit]):
            shutil.copy(path, f"{out_dir}/{cls}/{i}.jpg")
        print(cls, len(files[:limit]))

collect(f"{SRC}/Training", "data/train", 500)
collect(f"{SRC}/Test", "data/val", 150)

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

IMG = 224
train_ds = keras.utils.image_dataset_from_directory(
    "data/train", image_size=(IMG, IMG), batch_size=32, shuffle=True)
val_ds = keras.utils.image_dataset_from_directory(
    "data/val", image_size=(IMG, IMG), batch_size=32)

class_names = train_ds.class_names
print(class_names)

AUTOTUNE = tf.data.AUTOTUNE
```

```
aug = keras.Sequential([layers.RandomFlip("horizontal"), layers.RandomRotation(0.1)])
train_ds = train_ds.map(lambda x, y: (aug(x, training=True), y)).prefetch(AUTOTUNE)
val_ds = val_ds.prefetch(AUTOTUNE)

base = keras.applications.MobileNetV2(
    input_shape=(IMG, IMG, 3), include_top=False, weights="imagenet")
base.trainable = False

inputs = keras.Input(shape=(IMG, IMG, 3))
x = layers.Rescaling(1./127.5, offset=-1)(inputs)
x = base(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.2)(x)
outputs = layers.Dense(len(class_names), activation="softmax")(x)
model = keras.Model(inputs, outputs)

model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

model.fit(train_ds, validation_data=val_ds, epochs=5)

converter = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_model = converter.convert()

with open("model.tflite", "wb") as f:
    f.write(tflite_model)

with open("labels.txt", "w") as f:
    f.write("\n".join(class_names))

from google.colab import files
files.download("model.tflite")
files.download("labels.txt")
```

7 ResultActivity.java

7.1 Purpose

ResultActivity displays the food classification result to the user.

It receives the predicted label, confidence score, and selected image from CameraActivity.

Complete Source Code:

Listing 7: ResultActivity.java

```
package com.example.foodai;

import android.graphics.Bitmap;
import android.graphics.BitmapFactory;
import android.net.Uri;
import android.os.Bundle;
import android.widget.ImageButton;
import android.widget.ImageView;
import android.widget.TextView;

import androidx.appcompat.app.AppCompatActivity;

import java.util.HashMap;
import java.util.Locale;
import java.util.Map;

public class ResultActivity extends AppCompatActivity {

    public static final String EXTRA_IMAGE_PATH = "image_path";
    public static final String EXTRA_IMAGE_URI = "image_uri";
    public static final String EXTRA_LABEL = "label";
    public static final String EXTRA_CONFIDENCE = "confidence";

    // Nutrition per 100g: {kcal, carbs, protein, fat}
```

```
    private static final Map<String, float[]> NUTRITION = new HashMap<>();
    static {
        NUTRITION.put("orange",     new float[]{52f, 13f, 1.0f, 0.2f});
        NUTRITION.put("apple",      new float[]{52f, 14f, 0.3f, 0.2f});
        NUTRITION.put("banana",     new float[]{89f, 23f, 1.1f, 0.3f});
        NUTRITION.put("strawberry", new float[]{32f, 7.7f, 0.7f, 0.3f});
        NUTRITION.put("grape",      new float[]{69f, 18f, 0.7f, 0.2f});
        NUTRITION.put("mango",      new float[]{60f, 15f, 0.8f, 0.4f});
        NUTRITION.put("tomato",     new float[]{18f, 3.9f, 0.9f, 0.2f});
        NUTRITION.put("potato",     new float[]{77f, 17f, 2.0f, 0.1f});
        NUTRITION.put("pizza",      new float[]{266f, 33f, 11f, 10f});
        NUTRITION.put("burger",     new float[]{295f, 24f, 17f, 14f});
    }

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_result);

        ImageButton btnBack = findViewById(R.id.btnBack);
        ImageView imgFood = findViewById(R.id.imgFood);
        TextView tvFoodName = findViewById(R.id.tvFoodName);
        TextView tvConfidence = findViewById(R.id.tvConfidence);
        TextView tvCalories = findViewById(R.id.tvCalories);
        TextView tvCarbs = findViewById(R.id.tvCarbs);
        TextView tvProtein = findViewById(R.id.tvProtein);
        TextView tvFat = findViewById(R.id.tvFat);

        btnBack.setOnClickListener(v -> finish());

        String label = getIntent().getStringExtra(EXTRA_LABEL);
        float confidence = getIntent().getFloatExtra(EXTRA_CONFIDENCE, 0f);
        String imagePath = getIntent().getStringExtra(EXTRA_IMAGE_PATH);
        String imageUri = getIntent().getStringExtra(EXTRA_IMAGE_URI);

        // Photo dikhana (camera se ya gallery se)
        if (imagePath != null) {
            Bitmap bmp = BitmapFactory.decodeFile(imagePath);
            if (bmp != null) imgFood.setImageBitmap(bmp);
        } else if (imageUri != null) {
            imgFood.setImageURI(Uri.parse(imageUri));
        }

        if (label == null) label = "Unknown";
        tvFoodName.setText(capitalize(label));
        tvConfidence.setText("Prediction confidence  " + Math.round(confidence * 100) + "%");

        float[] n = NUTRITION.get(label.toLowerCase(Locale.ROOT));
        if (n != null) {
            tvCalories.setText(fmt(n[0]) + " kcal");
            tvCarbs.setText(fmt(n[1]) + " g");
            tvProtein.setText(fmt(n[2]) + " g");
            tvFat.setText(fmt(n[3]) + " g");
        } else {
            tvCalories.setText("--");
            tvCarbs.setText("--");
            tvProtein.setText("--");
            tvFat.setText("--");
        }
    }

    private String capitalize(String s) {
        s = s.replace('_', ' ');
        return s.isEmpty() ? s : Character.toUpperCase(s.charAt(0)) + s.substring(1);
    }

    private String fmt(float v) {
        return (v == Math.floor(v))
                ? String.format(Locale.US, "%.0f", v)
                : String.format(Locale.US, "%.1f", v);
    }
}
```

7.2 Detailed Explanation

The activity receives prediction information through Intent extras.

The predicted food label is displayed on the screen.

The confidence score is converted into percentage format for easier understanding.

The application also displays nutritional information such as:

- Calories
- Carbohydrates
- Protein
- Fat

These nutritional values are obtained from the application's local nutrition mapping.

8 AndroidManifest.xml

Purpose:

The manifest defines application activities, permissions, and other important Android configuration details.

Complete Code:

Listing 8: AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-permission android:name="android.permission.CAMERA" />

    <uses-feature
        android:name="android.hardware.camera"
        android:required="false" />

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.FoodAI">
        <activity
            android:name=".CameraActivity"
            android:exported="false" />
        <activity
            android:name=".ResultActivity"
            android:exported="false" />
        <activity
            android:name=".MainActivity"
            android:exported="false" />
        <activity
            android:name=".LoginActivity"
            android:exported="false"/>
        <activity
            android:name=".SplashActivity"
            android:exported="true"
            android:windowSoftInputMode="adjustResize">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

```

```
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

    </application>

</manifest>
```

Explanation:

The CAMERA permission allows the application to access the device camera.

Activities are registered in the manifest so Android can navigate between application screens.

9 Gradle Dependencies

9.1 Purpose

Gradle dependencies provide the external libraries required by FoodAI.

The project uses CameraX for image capture and TensorFlow Lite for on-device inference.

Complete Dependencies:

Listing 9: app/build.gradle

```
plugins {
    alias(libs.plugins.android.application)
}

android {
    namespace 'com.example.foodai'
    compileSdk {
        version = release(37)
    }

    defaultConfig {
        applicationId "com.example.foodai"
        minSdk 24
        targetSdk 37
        versionCode 1
        versionName "1.0"

        testInstrumentationRunner "androidx.test.runner.AndroidJUnitRunner"
    }

    buildTypes {
        release {
            optimization {
                enable false
            }
        }
    }
    compileOptions {
        sourceCompatibility JavaVersion.VERSION_11
        targetCompatibility JavaVersion.VERSION_11
    }
}

dependencies {
        // CameraX
        implementation 'androidx.camera:camera-core:1.3.4'
        implementation 'androidx.camera:camera-camera2:1.3.4'
        implementation 'androidx.camera:camera-lifecycle:1.3.4'
        implementation 'androidx.camera:camera-view:1.3.4'

        // TensorFlow Lite
    implementation 'org.tensorflow:tensorflow-lite:2.17.0'

    implementation libs.activity.ktx
    implementation libs.appcompat
```

```
    implementation libs.constraintlayout
    implementation libs.material
    testImplementation libs.junit
    androidTestImplementation libs.espresso.core
    androidTestImplementation libs.ext.junit
}
```

Explanation:

CameraX dependencies provide camera preview, capture, and lifecycle functionality.

The TensorFlow Lite dependency provides the Interpreter required to execute the trained machine learning model.

10 XML Layout Files

XML files define the visual interface of the FoodAI application.

10.1 Main Screen Layout

Listing 10: activity_main.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@drawable/bg_gradient_top">

    <!-- Decorative leaves -->
    <ImageView
        android:layout_width="90dp"
        android:layout_height="90dp"
        android:layout_gravity="top|start"
        android:layout_marginStart="6dp"
        android:layout_marginTop="40dp"
        android:alpha="0.18"
        android:rotation="-25"
        android:src="@drawable/ic_leaf" />

    <ImageView
        android:layout_width="80dp"
        android:layout_height="80dp"
        android:layout_gravity="top|end"
        android:layout_marginTop="40dp"
        android:layout_marginEnd="4dp"
        android:alpha="0.15"
        android:rotation="25"
        android:src="@drawable/ic_leaf" />

    <ImageView
        android:layout_width="60dp"
        android:layout_height="60dp"
        android:layout_gravity="bottom|start"
        android:layout_marginStart="10dp"
        android:layout_marginBottom="20dp"
        android:alpha="0.7"
        android:rotation="-20"
        android:src="@drawable/ic_leaf"
        app:tint="@color/green_check" />

    <ImageView
        android:layout_width="50dp"
        android:layout_height="50dp"
        android:layout_gravity="bottom|end"
        android:layout_marginEnd="10dp"
        android:layout_marginBottom="24dp"
        android:alpha="0.6"
        android:rotation="30"
```

```
        android:src="@drawable/ic_leaf"
        app:tint="@color/green_check" />

    <!-- Main white card -->

    <!-- Home indicator -->

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:layout_marginStart="20dp"
        android:layout_marginEnd="18dp"
        android:layout_marginBottom="0dp"
        android:background="@drawable/bg_card"
        android:elevation="8dp"
        android:orientation="vertical"
        android:padding="18dp">

        <!-- Camera icon with small leaves -->
        <LinearLayout
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_gravity="center_horizontal"
            android:gravity="bottom"
            android:orientation="horizontal">

            <ImageView
                android:layout_width="16dp"
                android:layout_height="16dp"
                android:layout_marginBottom="14dp"
                android:alpha="0.7"
                android:rotation="-30"
                android:src="@drawable/ic_leaf"
                app:tint="@color/green_check" />

            <FrameLayout
                android:layout_width="64dp"
                android:layout_height="64dp"
                android:layout_marginStart="10dp"
                android:layout_marginEnd="10dp"
                android:background="@drawable/bg_circle_light">

                <ImageView
                    android:layout_width="28dp"
                    android:layout_height="28dp"
                    android:layout_gravity="center"
                    android:src="@drawable/ic_camera" />
            </FrameLayout>

            <ImageView
                android:layout_width="16dp"
                android:layout_height="16dp"
                android:layout_marginBottom="14dp"
                android:alpha="0.7"
                android:rotation="30"
                android:src="@drawable/ic_leaf"
                app:tint="@color/green_check" />
        </LinearLayout>

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="14dp"
            android:gravity="center"
            android:text="@string/scan_title"
            android:textColor="@color/text_dark"
            android:textSize="24sp"
            android:textStyle="bold" />

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="6dp"
            android:gravity="center"
            android:text="@string/scan_subtitle"
```

```
            android:textColor="@color/text_grey"
            android:textSize="12sp" />

        <!-- Row 1: Camera -->
        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="18dp"
            android:background="@drawable/bg_row"
            android:gravity="center_vertical"
            android:orientation="horizontal"
            android:paddingStart="14dp"
            android:paddingTop="10dp"
            android:paddingEnd="14dp"
            android:paddingBottom="10dp">

            <ImageView
                android:layout_width="22dp"
                android:layout_height="22dp"
                android:src="@drawable/ic_camera" />

            <LinearLayout
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_marginStart="14dp"
                android:layout_weight="1"
                android:orientation="vertical">

                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="@string/camera_quality"
                    android:textColor="@color/text_dark"
                    android:textSize="13sp"
                    android:textStyle="bold" />

                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="@string/camera_quality_sub"
                    android:textColor="@color/text_grey"
                    android:textSize="10sp" />
            </LinearLayout>

            <ImageView
                android:layout_width="20dp"
                android:layout_height="20dp"
                android:src="@drawable/ic_check_circle" />
        </LinearLayout>

        <!-- Row 2: Internet -->
        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="8dp"
            android:background="@drawable/bg_row"
            android:gravity="center_vertical"
            android:orientation="horizontal"
            android:paddingStart="14dp"
            android:paddingTop="10dp"
            android:paddingEnd="14dp"
            android:paddingBottom="10dp">

            <ImageView
                android:layout_width="22dp"
                android:layout_height="22dp"
                android:src="@drawable/ic_wifi" />

            <LinearLayout
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_marginStart="14dp"
                android:layout_weight="1"
                android:orientation="vertical">

                <TextView
```

```
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="@string/internet_speed"
                    android:textColor="@color/text_dark"
                    android:textSize="13sp"
                    android:textStyle="bold" />

                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="@string/internet_speed_sub"
                    android:textColor="@color/text_grey"
                    android:textSize="10sp" />
            </LinearLayout>

            <ImageView
                android:layout_width="20dp"
                android:layout_height="20dp"
                android:src="@drawable/ic_check_circle" />
        </LinearLayout>

        <!-- Row 3: Lighting -->
        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="8dp"
            android:background="@drawable/bg_row"
            android:gravity="center_vertical"
            android:orientation="horizontal"
            android:paddingStart="14dp"
            android:paddingTop="10dp"
            android:paddingEnd="14dp"
            android:paddingBottom="10dp">

            <ImageView
                android:layout_width="22dp"
                android:layout_height="22dp"
                android:src="@drawable/ic_bulb" />

            <LinearLayout
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_marginStart="14dp"
                android:layout_weight="1"
                android:orientation="vertical">

                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="@string/well_lit"
                    android:textColor="@color/text_dark"
                    android:textSize="13sp"
                    android:textStyle="bold" />

                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="@string/well_lit_sub"
                    android:textColor="@color/text_grey"
                    android:textSize="10sp" />
            </LinearLayout>

            <ImageView
                android:layout_width="20dp"
                android:layout_height="20dp"
                android:src="@drawable/ic_check_circle" />
        </LinearLayout>

        <!-- Primary button -->
        <com.google.android.material.button.MaterialButton
            android:id="@+id/btnScan"
            android:layout_width="match_parent"
            android:layout_height="52dp"
            android:layout_marginTop="20dp"
            android:text="@string/lets_scan"
            android:textAllCaps="false"
```

```
            android:textColor="@color/white"
            android:textSize="15sp"
            android:textStyle="bold"
            app:backgroundTint="@color/green_dark"
            app:cornerRadius="26dp" />

        <!-- OR divider -->
        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="8dp"
            android:layout_marginBottom="8dp"
            android:gravity="center_vertical"
            android:orientation="horizontal">

            <View
                android:layout_width="0dp"
                android:layout_height="1dp"
                android:layout_marginStart="80dp"
                android:layout_weight="1"
                android:background="@color/divider" />

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_marginStart="8dp"
                android:layout_marginEnd="8dp"
                android:text="@string/or_text"
                android:textColor="@color/text_grey"
                android:textSize="10sp" />

            <View
                android:layout_width="0dp"
                android:layout_height="1dp"
                android:layout_marginEnd="80dp"
                android:layout_weight="1"
                android:background="@color/divider" />
        </LinearLayout>

        <!-- Secondary button -->
        <com.google.android.material.button.MaterialButton
            android:id="@+id/btnManual"
            style="?attr/materialButtonOutlinedStyle"
            android:layout_width="match_parent"
            android:layout_height="46dp"
            android:text="@string/log_manually"
            android:textAllCaps="false"
            android:textColor="@color/text_dark"
            android:textSize="13sp"
            app:cornerRadius="23dp"
            app:icon="@drawable/ic_edit"
            app:iconGravity="textStart"
            app:iconSize="16dp"
            app:iconTint="@color/green_dark"
            app:strokeColor="#CFE0D6" />
    </LinearLayout>


</FrameLayout>
```

Explanation:

This layout defines the main screen, including the application title, food scanning button, and other interface elements.

10.2 Camera Screen Layout

Listing 11: activity_camera.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
    android:background="@color/black">

    <androidx.camera.view.PreviewView
        android:id="@+id/previewView"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />

    <!-- Scan frame -->
    <View
        android:layout_width="280dp"
        android:layout_height="280dp"
        android:layout_gravity="center"
        android:background="@drawable/bg_scan_frame" />

    <ImageButton
        android:id="@+id/btnBack"
        android:layout_width="42dp"
        android:layout_height="42dp"
        android:layout_gravity="top|start"
        android:layout_marginStart="16dp"
        android:layout_marginTop="40dp"
        android:background="@drawable/bg_circle_dark"
        android:contentDescription="Back"
        android:src="@drawable/ic_back" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="top|center_horizontal"
        android:layout_marginTop="100dp"
        android:background="@drawable/bg_pill_dark"
        android:paddingStart="16dp"
        android:paddingTop="8dp"
        android:paddingEnd="16dp"
        android:paddingBottom="8dp"
        android:text="@string/align_food"
        android:textColor="@color/white"
        android:textSize="13sp" />

    <!-- Bottom controls -->
    <RelativeLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_gravity="bottom"
        android:layout_marginBottom="36dp"
        android:paddingStart="24dp"
        android:paddingEnd="24dp">

        <LinearLayout
            android:id="@+id/btnGallery"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_alignParentStart="true"
            android:layout_centerVertical="true"
            android:gravity="center"
            android:orientation="vertical">

            <ImageView
                android:layout_width="44dp"
                android:layout_height="44dp"
                android:background="@drawable/bg_circle_dark"
                android:padding="10dp"
                android:src="@drawable/ic_gallery" />

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="@string/gallery"
                android:textColor="@color/white"
                android:textSize="11sp" />
        </LinearLayout>

        <ImageButton
            android:id="@+id/btnCapture"
            android:layout_width="80dp"
```

```
            android:layout_height="80dp"
            android:layout_centerHorizontal="true"
            android:background="@drawable/bg_capture_button"
            android:contentDescription="Capture"
            android:src="@drawable/ic_camera"
            app:tint="@color/white" />

        <LinearLayout
            android:id="@+id/btnFlash"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_alignParentEnd="true"
            android:layout_centerVertical="true"
            android:gravity="center"
            android:orientation="vertical">

            <ImageView
                android:layout_width="44dp"
                android:layout_height="44dp"
                android:background="@drawable/bg_circle_dark"
                android:padding="10dp"
                android:src="@drawable/ic_flash" />

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="@string/flash"
                android:textColor="@color/white"
                android:textSize="11sp" />
        </LinearLayout>
    </RelativeLayout>
</FrameLayout>
```

Explanation:

This layout contains the camera preview and controls required for capturing food images.

10.3 Result Screen Layout

Listing 12: activity_result.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@drawable/bg_gradient_top"
    android:fillViewport="true">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:paddingBottom="24dp">

        <ImageButton
            android:id="@+id/btnBack"
            android:layout_width="42dp"
            android:layout_height="42dp"
            android:layout_marginStart="16dp"
            android:layout_marginTop="40dp"
            android:background="@drawable/bg_circle_dark"
            android:contentDescription="Back"
            android:src="@drawable/ic_back" />

        <ImageView
            android:id="@+id/imgFood"
            android:layout_width="120dp"
            android:layout_height="120dp"
            android:layout_gravity="center_horizontal"
            android:layout_marginTop="8dp"
            android:background="@drawable/bg_nutrition_card"
            android:padding="4dp"
            android:scaleType="centerCrop" />
```

```
33
34          <TextView
35              android:id="@+id/tvFoodName"
36              android:layout_width="match_parent"
37              android:layout_height="wrap_content"
38              android:layout_marginTop="14dp"
39              android:gravity="center"
40              android:text="Orange"
41              android:textColor="@color/text_dark"
42              android:textSize="24sp"
43              android:textStyle="bold" />
44
45          <TextView
46              android:layout_width="match_parent"
47              android:layout_height="wrap_content"
48              android:gravity="center"
49              android:text="@string/ai_food_id"
50              android:textColor="@color/text_grey"
51              android:textSize="13sp" />
52
53          <TextView
54              android:id="@+id/tvConfidence"
55              android:layout_width="wrap_content"
56              android:layout_height="wrap_content"
57              android:layout_gravity="center_horizontal"
58              android:layout_marginTop="14dp"
59              android:background="@drawable/bg_pill_light"
60              android:drawableStart="@drawable/ic_sparkle"
61              android:drawablePadding="8dp"
62              android:paddingStart="24dp"
63              android:paddingTop="10dp"
64              android:paddingEnd="24dp"
65              android:paddingBottom="10dp"
66              android:text="Prediction confidence  96%"
67              android:textColor="@color/text_dark"
68              android:textSize="13sp" />
69
70          <TextView
71              android:layout_width="wrap_content"
72              android:layout_height="wrap_content"
73              android:layout_marginStart="24dp"
74              android:layout_marginTop="24dp"
75              android:text="@string/nutrition_estimate"
76              android:textColor="@color/text_dark"
77              android:textSize="15sp"
78              android:textStyle="bold" />
79
80          <!-- Row 1: Calories + Carbs -->
81          <LinearLayout
82              android:layout_width="match_parent"
83              android:layout_height="wrap_content"
84              android:layout_marginStart="20dp"
85              android:layout_marginTop="10dp"
86              android:layout_marginEnd="20dp"
87              android:orientation="horizontal">
88
89              <LinearLayout
90                  android:layout_width="0dp"
91                  android:layout_height="wrap_content"
92                  android:layout_marginEnd="6dp"
93                  android:layout_weight="1"
94                  android:background="@drawable/bg_nutrition_card"
95                  android:orientation="vertical"
96                  android:padding="14dp">
97
98                  <TextView
99                      android:layout_width="wrap_content"
100                     android:layout_height="wrap_content"
101                     android:drawableStart="@drawable/ic_fire"
102                     android:drawablePadding="6dp"
103                     android:text="@string/calories"
104                     android:textColor="@color/text_grey"
105                     android:textSize="12sp" />
106
107                 <TextView
108                     android:id="@+id/tvCalories"
```

```
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:layout_marginTop="6dp"
                    android:text="52 kcal"
                    android:textColor="@color/text_dark"
                    android:textSize="20sp"
                    android:textStyle="bold" />

                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="@string/per_100g"
                    android:textColor="@color/text_grey"
                    android:textSize="10sp" />
            </LinearLayout>

            <LinearLayout
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_marginStart="6dp"
                android:layout_weight="1"
                android:background="@drawable/bg_nutrition_card"
                android:orientation="vertical"
                android:padding="14dp">

                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:drawableStart="@drawable/ic_carbs"
                    android:drawablePadding="6dp"
                    android:text="@string/carbs"
                    android:textColor="@color/text_grey"
                    android:textSize="12sp" />

                <TextView
                    android:id="@+id/tvCarbs"
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:layout_marginTop="6dp"
                    android:text="13 g"
                    android:textColor="@color/text_dark"
                    android:textSize="20sp"
                    android:textStyle="bold" />

                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="@string/per_100g"
                    android:textColor="@color/text_grey"
                    android:textSize="10sp" />
            </LinearLayout>
        </LinearLayout>

        <!-- Row 2: Protein + Fat -->
        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginStart="20dp"
            android:layout_marginTop="12dp"
            android:layout_marginEnd="20dp"
            android:orientation="horizontal">

            <LinearLayout
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_marginEnd="6dp"
                android:layout_weight="1"
                android:background="@drawable/bg_nutrition_card"
                android:orientation="vertical"
                android:padding="14dp">

                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:drawableStart="@drawable/ic_dumbbell"
                    android:drawablePadding="6dp"
```

```
                    android:text="@string/protein"
                    android:textColor="@color/text_grey"
                    android:textSize="12sp" />

                <TextView
                    android:id="@+id/tvProtein"
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:layout_marginTop="6dp"
                    android:text="1.0 g"
                    android:textColor="@color/text_dark"
                    android:textSize="20sp"
                    android:textStyle="bold" />

                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="@string/per_100g"
                    android:textColor="@color/text_grey"
                    android:textSize="10sp" />
            </LinearLayout>

            <LinearLayout
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_marginStart="6dp"
                android:layout_weight="1"
                android:background="@drawable/bg_nutrition_card"
                android:orientation="vertical"
                android:padding="14dp">

                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:drawableStart="@drawable/ic_drop"
                    android:drawablePadding="6dp"
                    android:text="@string/fat"
                    android:textColor="@color/text_grey"
                    android:textSize="12sp" />

                <TextView
                    android:id="@+id/tvFat"
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:layout_marginTop="6dp"
                    android:text="0.2 g"
                    android:textColor="@color/text_dark"
                    android:textSize="20sp"
                    android:textStyle="bold" />

                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="@string/per_100g"
                    android:textColor="@color/text_grey"
                    android:textSize="10sp" />
            </LinearLayout>
        </LinearLayout>

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginStart="20dp"
            android:layout_marginTop="16dp"
            android:layout_marginEnd="20dp"
            android:background="@drawable/bg_pill_light"
            android:drawableStart="@drawable/ic_info"
            android:drawablePadding="10dp"
            android:padding="14dp"
            android:text="@string/nutrition_note"
            android:textColor="@color/text_dark"
            android:textSize="11sp" />
    </LinearLayout>
</ScrollView>
```

Explanation:

This layout displays the predicted food name, confidence score, image, and nutritional values.

10.4 Screenshots Required for the Final Report

The following screenshots demonstrate the implemented features of the FoodAI Android application.

Mini Project Structure

1. **Android Studio Project Structure**

   Insert a screenshot of the Android Studio project tree showing relevant Java source files, XML layouts, and model or asset files.

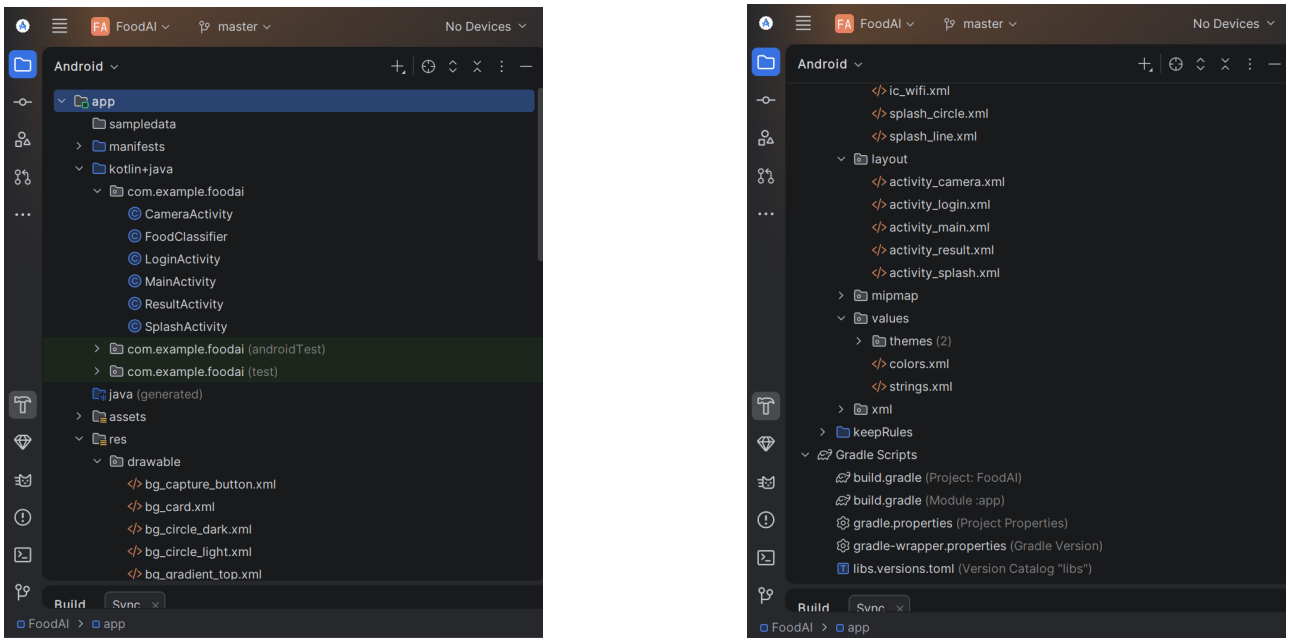


Figure 1: FoodAI Project Structure in Android Studio

2. **Gradle Configuration**

   Add a screenshot of the Gradle configuration showing the actual TensorFlow Lite or other inference dependency, if integrated.

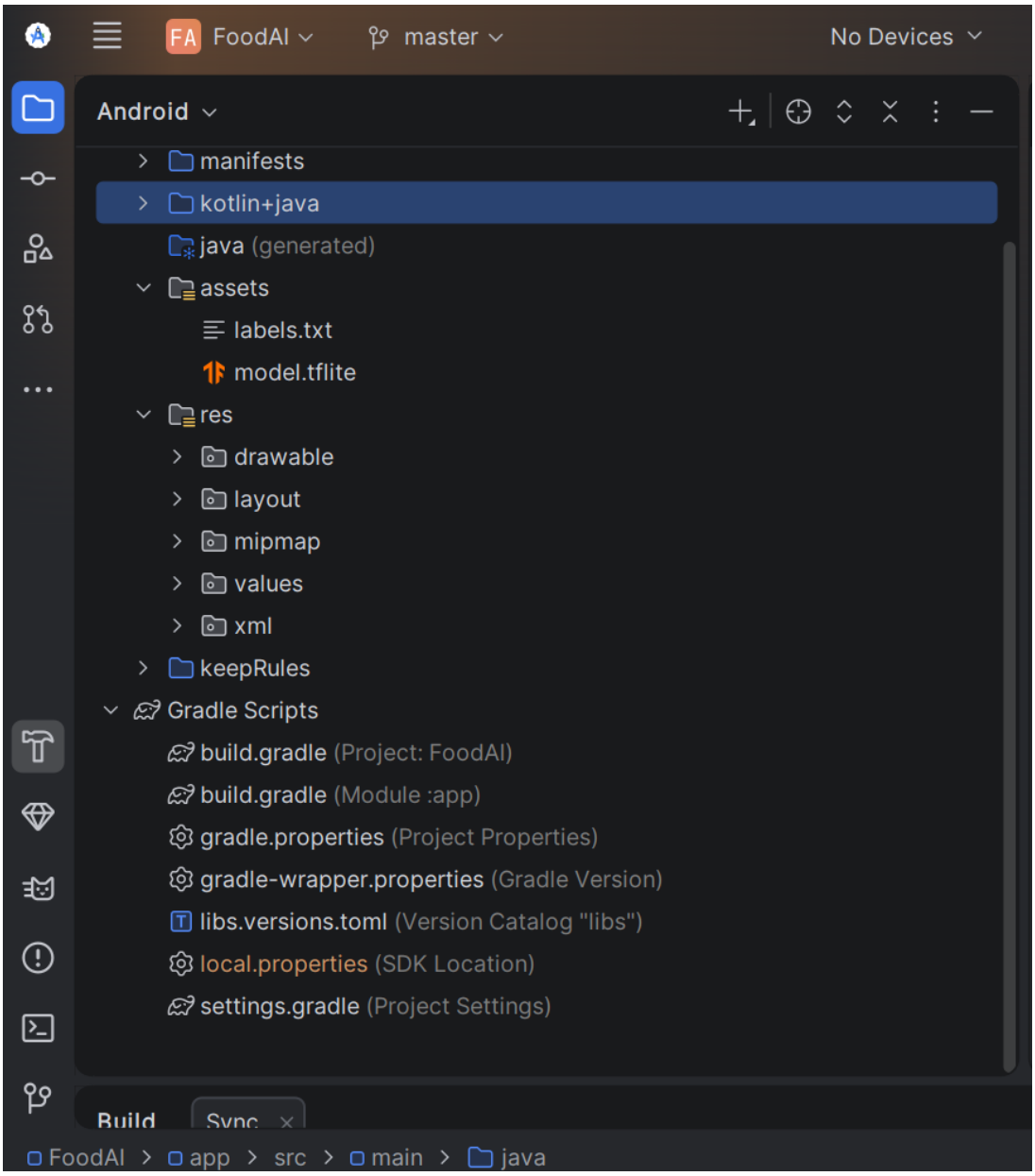


Figure 2: AI Inference Dependency Configuration

3. **Successful Build and Execution**

   Insert a screenshot showing that the application builds and runs successfully in Android Studio or on an Android emulator/device.

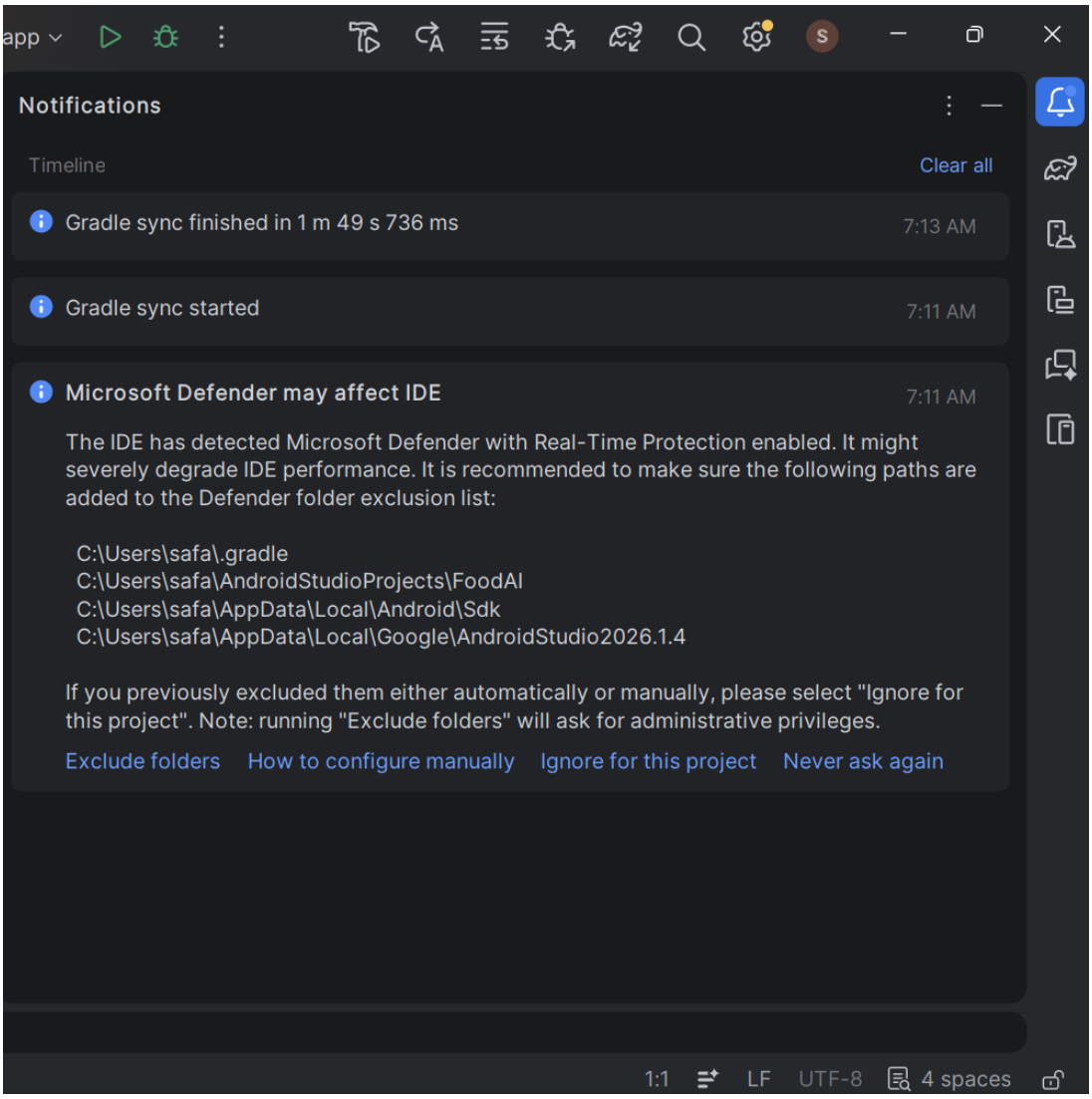


Figure 3: Successful FoodAI Application Execution

FoodAI Application Screenshots

1. **Splash Screen:**

   The splash screen is the first interface displayed when FoodAI starts. It introduces the application and provides a starting point for the user journey.

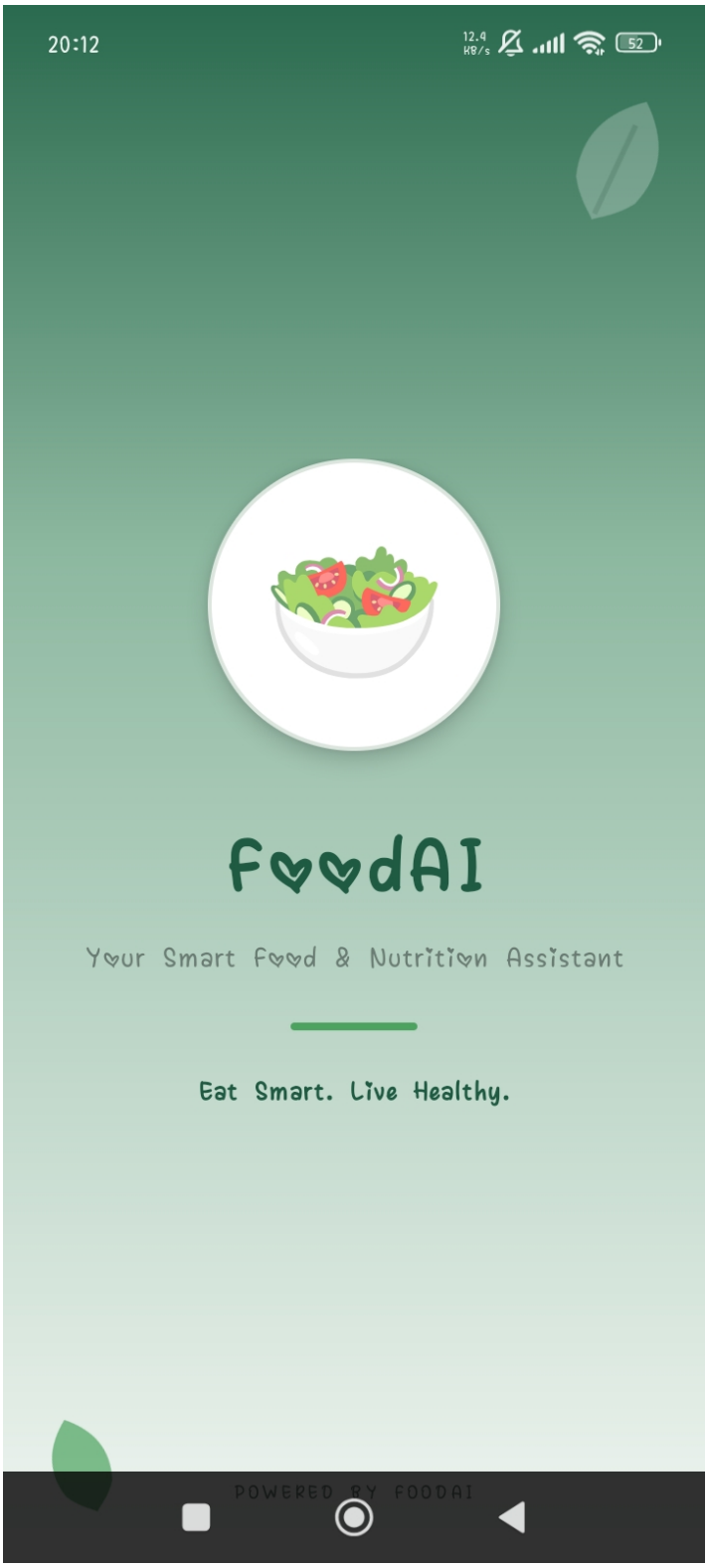


Figure 4: FoodAI Application Splash Screen

2. **Login and Sign-up Screens:**

   The login screen allows existing users to enter their credentials, while the sign-up screen provides an interface for new users to register, if authentication is implemented.

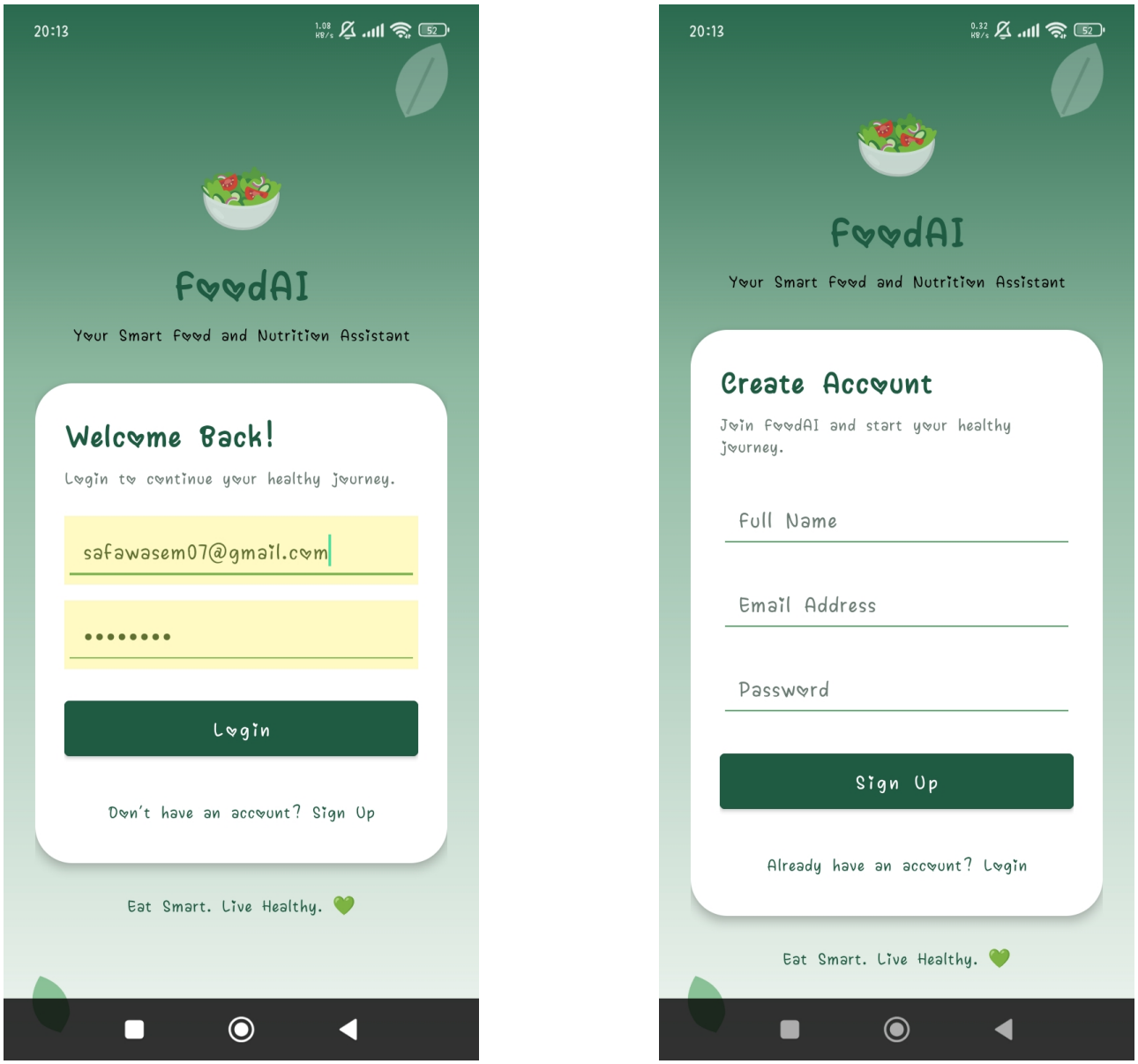


Figure 5: FoodAI Login and Sign-up Interfaces

3. **Home Screen and Navigation to Food Scanning:**

   The home screen introduces the main functionality of FoodAI. When the user clicks the scanning button, the application opens the food-scanning interface.

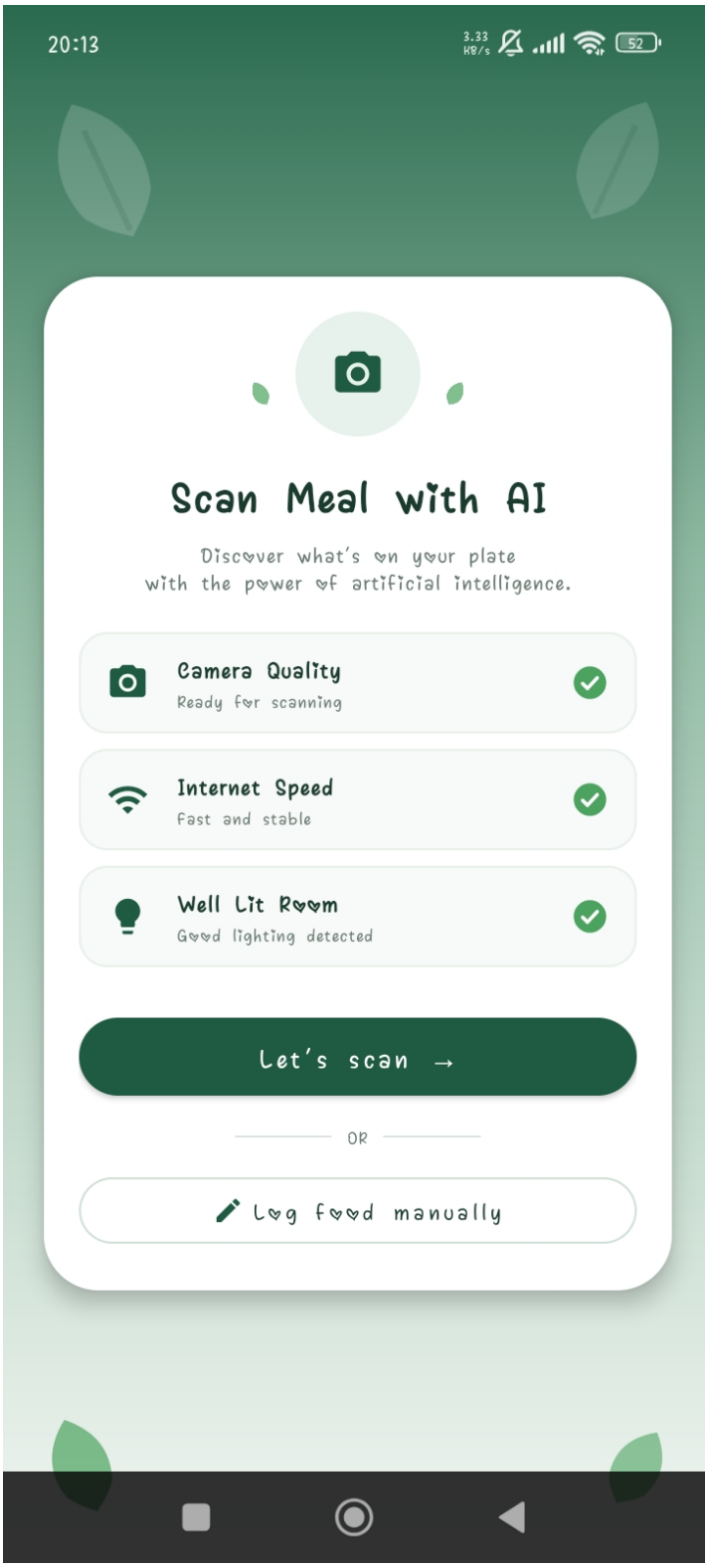


Figure 6: Goto Food Image Scanning Interface

4. **Food-Scanning Screen:**

   This screen allows users to choose a food image from the gallery or capture one using the camera. The selected image is then used for the food recognition process.

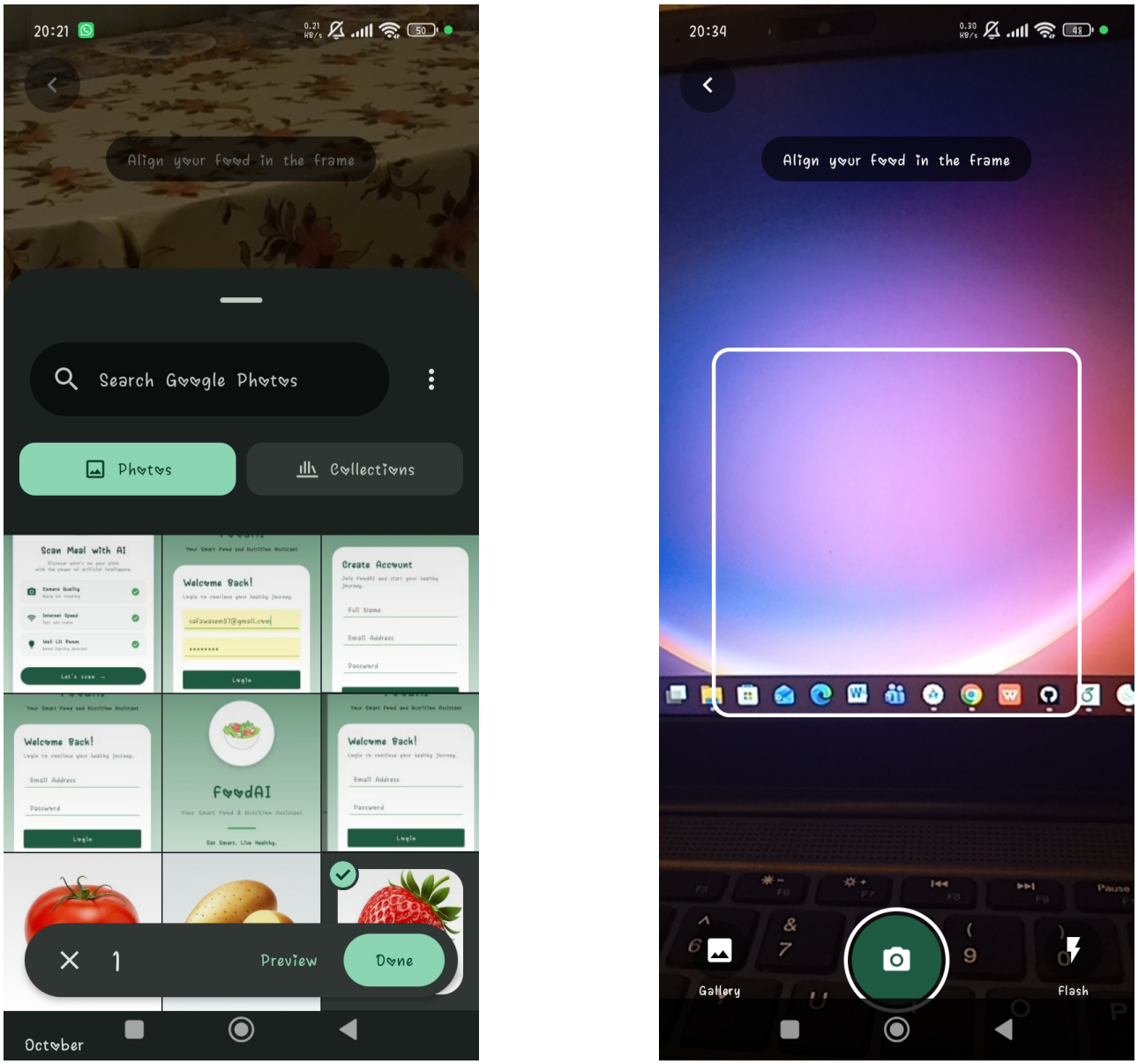


Figure 7: Food Image Selected from Gallery or Camera

5. **Selected or Captured Food Image:**

   The selected or captured image is displayed so the user can review it before analysis. The image may then be prepared according to the input requirements of the AI model.

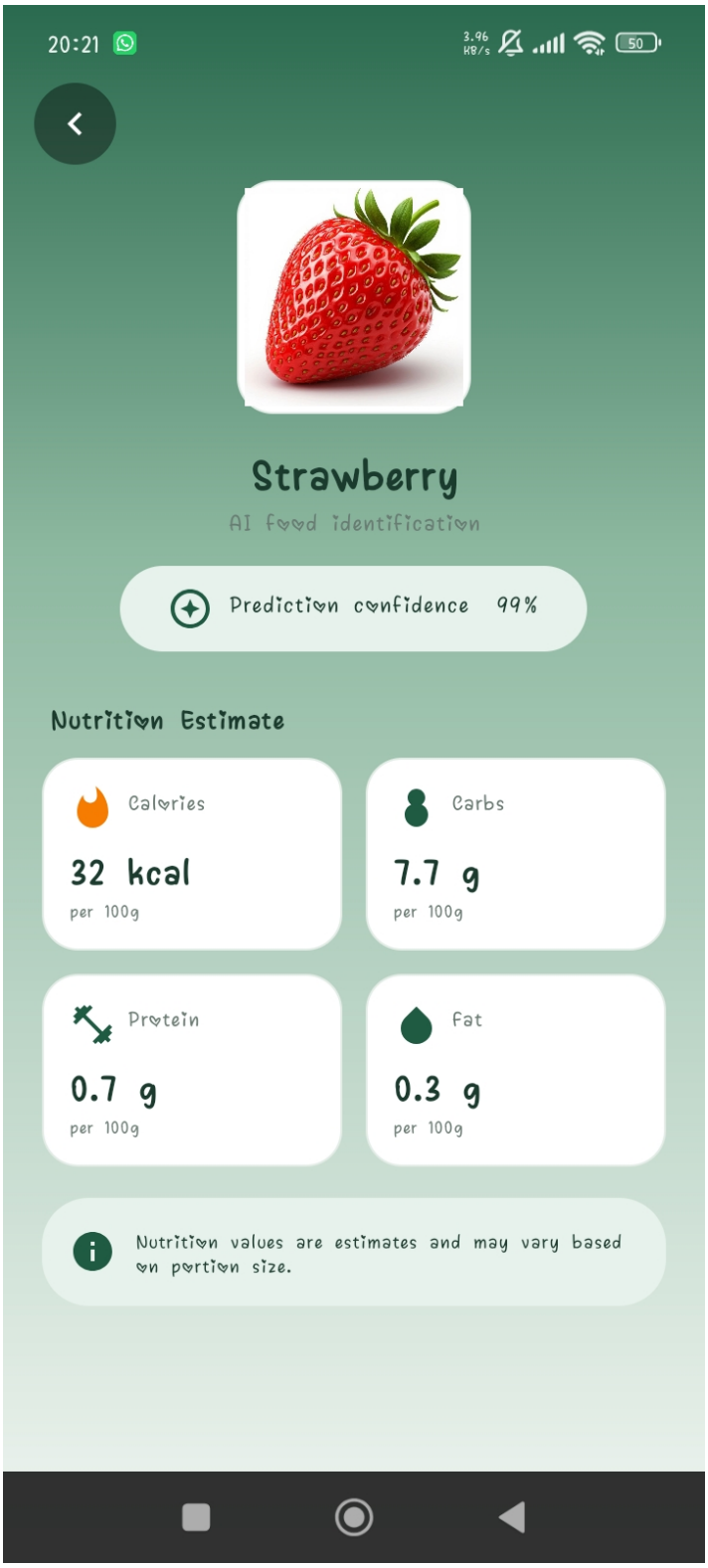


Figure 8: FoodAI AI Prediction Result

11 Training Results

Insert the actual training accuracy and loss graphs generated during model training.

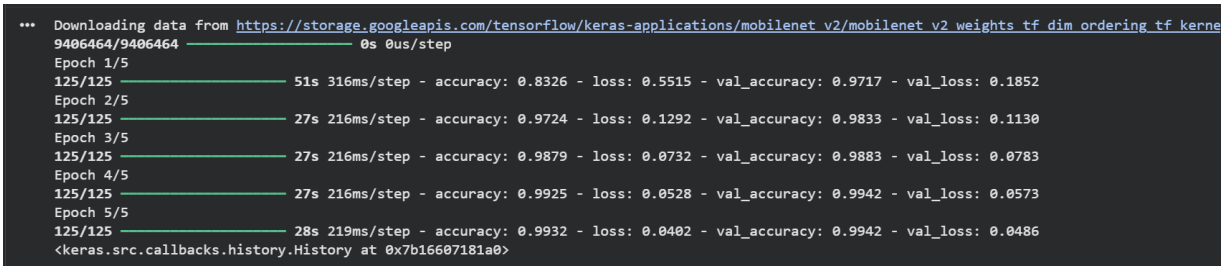


Figure 9: Training and Validation Accuracy

12 Testing and Discussion

The FoodAI application should be tested using different food images.

The testing process should verify:

1. Whether the application launches correctly.
2. Whether camera permission works.
3. Whether the user can capture an image.
4. Whether gallery image selection works.
5. Whether the TensorFlow Lite model loads.
6. Whether food classification produces a result.
7. Whether the result screen displays the predicted label and confidence score.
8. Whether nutritional information is displayed for supported food categories.

Actual test cases and observed results should be documented using screenshots.

13 Conclusion

FoodAI demonstrates the integration of machine learning into an Android application using TensorFlow Lite.

The project combines Java-based Android development, XML user interfaces, CameraX image acquisition, and on-device model inference.

The TensorFlow Lite Interpreter processes food images and generates prediction results.

The application demonstrates how trained machine learning models can be integrated into practical mobile applications.

14 References

1. Original GitHub Reference Repository:
   `https://github.com/Uzair390-Del/NumberGuessingAppTflite`
2. TensorFlow Lite Documentation:
   `https://www.tensorflow.org/lite`

3. Android Developers Documentation:
   https://developer.android.com/
4. CameraX Documentation:
   https://developer.android.com/media/camera/camerax